

RESULTS

Phase-3 RAG (Retrieval-Augmented Generation) — end-to-end proof (§11.4.108 / §11.4.5 / §11.4.69 / §11.4.147)

Date: 2026-07-07 · Track (T1/feature/helixllm-full-extension) · Resumed crashed §11.4.147 stream (prior session died mid-work, at “confirmed the off-by-one, fixing it” — a shared-session-limit crash, not a defect in this proof)

Verdict (honest, §11.4.6)

CAPABILITY PROVEN. A real, minimal RAG (embed → real-cosine top-k retrieve → grounded prompt → generate) pipeline was proven end-to-end against the **live coder LLM** (helixllm-coder, <http://localhost:18434>, read-only, never restarted/stopped — §11.4.122/§11.4.119) composed with a dedicated CPU embeddings service (HF TEI cpu-1.9, BAAI/bge-small-en-v1.5, dim 384) booted on its own port **18440** via the **containers submodule orchestrator** (§11.4.76, rootless podman §11.4.161). The §11.4.108 runtime signature is **GREEN-OK for both invented-fact queries**:

```
[RAG-RUNTIME-SIGNATURE] PASS top1=doc_codename(score=0.8852, ok=true) token
  answer="The internal codename for the 2027 HelixCode release is Boreal
[RAG-RUNTIME-SIGNATURE] PASS top1=doc_sidecar(score=0.8855, ok=true) token
  answer="The internal service-mesh sidecar used internally for HelixLLM
```

RED → GREEN runtime signature (§11.4.115 polarity, both queries)

Two facts invented for this proof (the coder cannot know them from training):

qkey	invented fact	fact-bearing doc
q1	“The internal codename for the 2027 HelixCode release is Borealis-9 .”	doc_codename
q2	“The internal service-mesh sidecar used internally for HelixLLM routing is called Quillfeather-7 .”	doc_sidecar

RED (RED_MODE=1, no retrieval context, 10_red_baseline.txt) — the live coder is asked the bare question with NO retrieved context: - q1: *“I don’t have any information about a 2027 HelixCode release or its internal codename, as this appears to be fictional or hypothetical.”* → does **NOT** contain Borealis-9. **RED-OK** (defect genuinely reproduced — the coder has no way to answer). - q2: *“The internal service-mesh sidecar used for HelixLLM routing is called “HelixProxy”.*” → the coder **hallucinates a wrong, different name** (HelixProxy ≠ Quillfeather-7) → does **NOT** contain the invented token. **RED-OK**.

GREEN (11_green_proof_{q1,q2}.txt) — full pipeline (embed corpus on real TEI vectors → real cosine top-2 retrieve → grounded system+user prompt, “answer ONLY from context” → generate on the SAME live coder): - q1 answer: *“The internal codename for the 2027 HelixCode release is Borealis-9.”* → **contains** Borealis-9. **GREEN-OK**. - q2 answer: *“The internal service-mesh sidecar used internally for HelixLLM routing is called Quillfeather-7.”* → **contains** Quillfeather-7. **GREEN-OK**.

RED-fail then GREEN-pass on the identical question, identical live model, is the unfakeable proof that generation is genuinely *grounded* in retrieval, not recalled from training or hardcoded.

Retrieval top-1 evidence (real cosine over real TEI embeddings, retrieval_{q1,q2}.json)

6-document corpus (2 fact-bearing + 4 topic-adjacent/unrelated distractors, so retrieval must genuinely discriminate — doc_cat, doc_revenue, doc_coffee, doc_ci). Full ranking, both queries:

```
qkey=q1  rank1=doc_codename(0.8852) rank2=doc_ci(0.7166) rank3=doc_sidecar
         rank4=doc_coffee(0.4577)   rank5=doc_revenue(0.3922) rank6=doc_cat
[RETRIEVAL-CHECK] PASS: real-cosine top1=doc_codename == expected doc_code
```

```
qkey=q2  rank1=doc_sidecar(0.8855) rank2=doc_codename(0.6819) rank3=doc_ci
         rank4=doc_coffee(0.4619)   rank5=doc_revenue(0.4114) rank6=doc_cat
[RETRIEVAL-CHECK] PASS: real-cosine top1=doc_sidecar == expected doc_sidec
```

Ranking is a pure sort. Slice over real cosine similarities computed from real TEI-served vectors (main.go:cmdRetrieve) — never a hardcoded pick; both fact-bearing documents rank strictly above every distractor with a wide margin (≥ 0.16 over the runner-up).

Self-validation — the analyzer is non-bluff

(§11.4.107(10), 12_self_validation_{q1,q2}.txt)

For each query, the SAME analyzer (analyze() in main.go) is run against one golden-good fixture (the real captured retrieval + generation above) and three deliberately-degraded golden-bad fixtures:

fixture	q1 verdict	q2 verdict
golden-good (real captured RAG output)	PASS	PASS
golden-bad: no-fact answer (“I don’t know...”)	FAIL (correctly)	FAIL (correctly)
golden-bad: wrong document retrieved (top-1 swapped to a distractor)	FAIL (correctly)	FAIL (correctly)
golden-bad: empty answer	FAIL (correctly)	FAIL (correctly)

[SELF-VALIDATION] PASS: analyzer PASSes golden-good and FAILs all golden-bad fixtures for both qkeys — an analyzer that PASSED any golden-bad fixture would itself be a bluff gate (§11.4.107(10)); it does not.

Container lifecycle (§11.4.119 single-resource-owner, §11.4.122 coder untouched)

- Booted phase3rag_tei-rag_1 (HF TEI cpu-1.9, BAAI/bge-small-en-v1.5) on its **own** port **18440** via digital.vasic.containers compose orchestrator (rootless podman, 20_boot.txt, 21_health.txt — health OK after 2 polls, 24_container_state.txt).
- helixllm-coder (the live coder at :18434) verified running before and after, **never** restarted/stopped (00_preflight.txt, 29b_post_teardown.txt).
- Sibling Phase-3 lanes on ports 18435–18439 verified free/untouched throughout (00_preflight.txt, 29b_post_teardown.txt).
- Torn down single-owner (29_teardown.txt: DOWN-OK); post-teardown confirms zero phase3rag_* containers, coder still up, sibling ports still free (29b_post_teardown.txt).
- An unrelated migrate-it-shared (postgres) container was observed running on the same shared host during this run (another project’s work, §11.4.174) — read-only observation only, never touched.

Honesty notes (§11.4.6 / §11.4.147 / §11.4.102)

1. **Resumed a crashed §11.4.147 stream, not started fresh.** The preserved partial state (`docs/qa/phase3_rag_20260707/*.txt/*.json` + the harness at `submodules/helix_llm/docs/qa/phase3_rag_20260707/harness/`) already showed a complete prior successful run (RED-OK/GREEN-OK/self-validation-PASS for both queries, clean teardown) dated ~11:42 UTC. `main.go` was inspected end-to-end for the “off-by-one” the crashed session referenced (`cmdRetrieve` top-1 indexing, `cmdGreen` top-2 context-stuffing loop bounds, golden-bad-wrong-retrieval index-0 substitution) — no off-by-one is present in the delivered code; the retrieval ranking, top-N context stuffing, and self-validation index handling are all correct. Rather than relying on evidence from before this session started, **the full pipeline was re-run in THIS session (§11.4.98 re-runability proof)** to obtain first-hand, freshly-witnessed evidence — the files under `docs/qa/phase3_rag_20260707/` now reflect the 2026-07-07T16:30Z run captured by this agent, superseding the earlier 11:42Z run byte-for-byte (same code, same result).
2. **Transient host-resource hiccup on the first re-run attempt, investigated (§11.4.102) not guessed.** The first attempt in this session hit `rootlessport fork/exec /proc/self/exe: resource temporarily unavailable` during `podman-compose up` (evidence: `22_teilogs.txt` (empty — container never started), `28_teardown_failed.txt`, `90_blocked.txt`, all preserved as honest transient-failure evidence, not deleted). Before retrying, host state was verified FACT-checked, not assumed transient: load average 3.86 (moderate), 251 GiB RAM with 226 GiB available, `threads-max` headroom 4502/2,056,076, only 3 network namespaces, 543 GB disk free, zero leaked `phase3rag_*` containers or `rootlessport` processes from this project. All resources healthy → retry was the safe, bounded, evidence-backed decision (§11.4.101), not a blind hope-it-passes loop. The retry succeeded cleanly end-to-end (`20_boot.txt` → `29b_post_teardown.txt` this run).
3. **No off-by-one bug survives in the shipped harness.** Reviewed `cmdRetrieve` (sorts by real cosine descending, `ranked[0]` is top-1 — no off-by-one), `cmdGreen` (for `i := 0; i < topN; i++` stuffs exactly the top-topN docs, numbered `i+1` for the human-readable context list — correct), and `cmdSelfValidate`’s golden-bad-wrong-retrieval mutation (`wrongRF.Ranked[0]` — correctly targets the top-1 slot). All indices are 0-based and consistent throughout.
4. Both the empty-corpus/off-by-one investigation and the transient-resource investigation are captured findings, not silent assumptions.

Reproduce

```
cd submodules/helix_llm/docs/qa/phase3_rag_20260707/harness && ./run_proof
```

Builds the harness (containers-submodule replace directive), boots its OWN TEI lane on port 18440 (config-injected — TEI_HOST_PORT/TEI_MEM_LIMIT/TEI_CPUS/TEI_MODEL_ID/HEALTH_TIMEOUT env- tunable), asks the live coder the RED baseline, embeds the 6-doc corpus + both queries, retrieves + checks top-1 + grounds + generates + analyzes for both qkeys, self-validates the analyzer, tears down single-owner, and leaves helixllm-coder + sibling ports 18435-18439 untouched. harness/phase3rag.bin is a build artefact (gitignored §11.4.30; regenerate via `go build .` or by re-running the script).

Composition

§11.4.76 (containers submodule) · §11.4.161 (rootless) · §11.4.108 (runtime signature) · §11.4.107(10) (self-validated analyzer) · §11.4.115 (RED-first polarity) · §11.4.119 (single-owner teardown) · §11.4.122 (coder untouched) · §11.4.6 (honest transient-failure investigation, no off-by-one guessed away) · §11.4.98 (re-runability proof) · §11.4.147 (conductor completion of the crashed subagent's stream — no work lost, partial state investigated + superseded with fresh evidence, never blindly discarded or blindly trusted) · §11.4.150 (deep-research note, 01_research_note.txt) · §11.4.174 (unrelated migrate-it-shared host-mate observed, never touched).

Sources verified (§11.4.99)

- HuggingFace Text Embeddings Inference (TEI) /v1/embeddings OpenAI-compatible endpoint — reused, proven lane from the sibling Phase-3 embeddings proof (docs/qa/phase3_embeddings_20260706/RESULTS.md, 2026-07-06/07).
- Retrieval-augmented generation “stuff”/“grounded QA” pattern (embed → top-k cosine retrieve → stuff context into prompt → generate) — standard RAG architecture pattern, no external doc fetch required for this minimal-harness proof; NO external solution found – original work for the harness implementation itself (§11.4.8).